



RAJALAKSHMI INSTITUTE OF TECHNOLOGY

Kuthambakkam, Chennai – 600124

CS23431-DESIGN AND ANALYSIS OF ALGORITHMS

LAB MANUAL

Name of the student :
Register Number :
DEPT. & SECTION :
LAB SEMESTER / YEAR : IV/II

1.Implement Linear Search. Determine the time required to search for an element. Repeat the experiment for different values of n, the number of elements in the list to be searched and plot a graph of the time taken versus n.

AIM :

To write a C program to implement Linear Search. Determine the time required to search for an element. Repeat the experiment for different values of n, the number of elements in the list to be searched and plot a graph of the time taken versus n

ALGORITHM:

Linear Search (Array A, Value x)

Step 1: Set i to 1

Step 2: if $i > n$ then go to step 7

Step 3: if $A[i] = x$ then go to step 6

Step 4: Set i to $i + 1$

Step 5: Go to Step 2

Step 6: Printf Element x Found at index i and go to step 8

Step 7: Printf element not found

Step 8: Exit

RESULT:

Thus, the C program for linear search algorithm was executed successfully and the output was verified.

2.Implement recursive Binary Search. Determine the time required to search an element. Repeat the experiment for different values of n, the number of elements in the list to be searched and plot a graph of the time taken versus n.

AIM :

To write a C program to implement recursive Binary Search. Determine the time required to search an element. Repeat the experiment for different values of n, the number of elements in the list to be searched and plot a graph of the time taken versus n.

ALGORITHM:

Step 1: Compare x with the middle element.

Step 2: If x matches with the middle element, we return the mid index.

Step 3: Else If x is greater than the mid element, then x can only lie in the right half subarray after the mid element. So, we recur for the right half.

Step 4: Else (x is smaller) recur for the left half.

RESULT:

Thus, the C program for recursive binary search algorithm was executed successfully and the output was verified.

3. Given a text txt [0...n-1] and a pattern pat [0...m-1], write a function search (char pat [], char txt []) that prints all occurrences of pat [] in txt []. You may assume that $n > m$.

AIM :

To write a C program to given a text txt [0...n-1] and a pattern pat [0...m-1], write a function search (char pat [], char txt []) that prints all occurrences of pat [] in txt []. You may assume that $n > m$.

ALGORITHM:

Step 1: We start the comparison of pat[j] with $j = 0$ with characters of the current window of text.

Step 2: We keep matching characters txt[i] and pat[j] and keep incrementing i and j while pat[j] and txt[i] keep **matching**.

Step 3: When we see a **mismatch**

1) We know that characters pat[0..j-1] match with txt[i-j...i-1]

(Note that j starts with 0 and increments it only when there is a match).

2) We also know (from the above definition) that lps[j-1] is the count of characters of pat[0..j-1] that are both proper prefix and suffix.

3) From the above two points, we can conclude that we do not need to match these lps[j-1] characters with txt[i-j...i-1] because we know that these characters will anyway match. Let us consider the above example to understand this.

RESULT:

Thus, the C program for Pattern search algorithm was executed successfully and the output was verified.

4.Sort a given set of elements using the insertion sort and Heap sort methods and determine the time required to sort the elements.

Repeat the experiment for different values of n , the number of elements in the list to be sorted and plot a graph of the time taken versus n .

AIM :

To write a c program to sort a given set of elements using the Insertion sort and Heap sort methods and determine the time required to sort the elements. Repeat the experiment for different values of n , the number of elements in the list to be sorted and plot a graph of the time taken versus n .

ALGORITHM (INSERTION SORT):

Step 1: Start the program

Step 2: Iterate from $\text{arr}[1]$ to $\text{arr}[N]$ over the array.

Step 3: Compare the current element (key) to its predecessor.

Step 4: If the key element is smaller than its predecessor, compare it to the elements before. Move the greater elements one position up to make space for the swapped element.

Step 5: Stop the program.

ALGORITHM (HEAP SORT):

Step 1: Start the program

Step 2: Build a max heap from the input data.

Step 3: At this point, the maximum element is stored at the root of the heap. Replace it with the last item of the heap followed by reducing the size of the heap by 1.

Finally, heapify the root of the tree.

Step 4: Repeat step 2 while the size of the heap is greater than 1.

Step 5: Stop the program

RESULT:

Thus, the c program for Pattern search algorithm was executed successfully and the output was verified.

5. Develop a program to implement graph traversal using Breadth First Search

AIM :

To write a c program to develop a program to implement graph traversal using Breadth First Search.

ALGORITHM:

Step 1: Consider the graph you want to navigate.

Step 2: Select any vertex in your graph, say v_1 , from which you want to traverse the graph.

Step 3: Examine any two data structure for traversing the graph Visited array (size of the graph)
Queue data structure

Step 4: Starting from the vertex, you will add to the visited array, and afterward, you will v_1 's adjacent vertices to the queue data structure.

Step 5: Now, using the FIFO concept, you must remove the element from the queue, put it into the visited array, and then return to the queue to add the adjacent vertices of the removed element.

Step 6: Repeat step 5 until the queue is not empty and no vertex left to bevisited

RESULT:

Thus, the c program for Pattern search algorithm was executed successfully and the output was verified

6. Develop a program to implement graph traversal using Depth First Search

AIM:

To write a C program to develop a program to implement graph traversal using Depth First Search.

ALGORITHM:

Step 1: Start by putting any one of the graph's vertices at the back of the queue.

Step 2: Now take the front item of the queue and add it to the visited list.

Step 3: Create a list of that vertex's adjacent nodes. Add those which are not within the visited list to the rear of the queue.

Step 4: Keep continuing steps two and three till the queue is empty. **Step 5:** Stop the program

RESULT:

Thus, the C program for depth first search algorithm was executed successfully and the output was verified.

7. From a given vertex in a weighted connected graph, develop a program to find the shortest paths to other vertices using Dijkstra's algorithm.

AIM :

To write a c program from a given vertex in a weighted connected graph, develop a program to find the shortest paths to other vertices using Dijkstra's algorithm.

ALGORITHM:

Step 1: Mark the source node with a current distance of 0 and the rest with infinity.

Step 2: Set the non-visited node with the smallest current distance as the current node.

Step 3: For each neighbor, N of the current node adds the current distance of the adjacent node with the weight of the edge connecting 0→1. If it is smaller than the current distance of Node, set it as the new current distance of N.

Step 4: Mark the current node 1 as visited.

RESULT:

Thus, the C program for depth first search algorithm was executed successfully and the output was verified.

8.Find the minimum cost spanning tree of a given undirected graph using Prim's algorithm.

AIM:

To write a C program to find the minimum cost spanning tree of a given undirected graph using Prim's algorithm.

ALGORITHM:

Step 1: Determine an arbitrary vertex as the starting vertex of the MST.

Step 2: Follow steps 3 to 5 till there are vertices that are not included in the MST (known as fringe vertex).

Step 3: Find edges connecting any tree vertex with the fringe vertices.

Step 4: Find the minimum among these edges.

Step 5: Add the chosen edge to the MST if it does not form any cycle.

Step 6: Return the MST and exit

RESULT:

Thus, the C program for depth first search algorithm was executed successfully and the output was verified.

9.Implement Floyd's algorithm for the All-Pairs- Shortest- Paths problem.

AIM :

To write a C program to Implement Floyd's algorithm for the All-Pairs-Shortest-Paths problem.

ALGORITHM:

Step 1: Initialize the solution matrix same as the input graph matrix as a first step.

Step 2: Then update the solution matrix by considering all vertices as an intermediate vertex.

Step 3: The idea is to one by one pick all vertices and updates all shortest paths which include the picked vertex as an intermediate vertex in the shortest path.

Step 4: When we pick vertex number k as an intermediate vertex, we already have considered vertices $\{0, 1, 2, \dots, k-1\}$ as intermediate vertices.

Step 5: For every pair (i, j) of the source and destination vertices respectively, there are two possible cases.

□ k is not an intermediate vertex in shortest path from i to j . We keep the value of $\text{dist}[i][j]$ as it is.

• k is an intermediate vertex in shortest path from i to j . We update the value of $\text{dist}[i][j]$ as $\text{dist}[i][k] + \text{dist}[k][j]$ if $\text{dist}[i][j] > \text{dist}[i][k] + \text{dist}[k][j]$

RESULT:

Thus, the C program for Floyd's algorithm was executed successfully and the output was verified.

10. Compute the transitive closure of a given directed graph using Warshall's algorithm

AIM :

To write a C program to compute the transitive closure of a given directed graph using Warshall's algorithm.

ALGORITHM:

Step 1: Start the program

Step 2: Instead of an integer resultant matrix ($\text{dist}[V][V]$ in floyd warshall), we can create a Boolean reach-ability matrix $\text{reach}[V][V]$ (we save space). The value $\text{reach}[i][j]$ will be 1 if j is reachable from i , otherwise 0.

Step 3: Instead of using arithmetic operations, we can use logical operations. For arithmetic operation '+', logical and '&&' is used, and for a '-', logical or '||' is used. (We save time by a constant factor. Time complexity is the same though)

Step 4: Stop the program

RESULT:

Thus, the C program for Warshall's algorithm was executed successfully and the output was verified.

11. Develop a program to find out the maximum and minimum numbers in a given list of n numbers using the divide and conquer technique.

AIM :

To write a C program to develop a program to find out the maximum and minimum numbers in a given list of n numbers using the divide and conquer technique.

ALGORITHM:

Step 1: Start the program.

Step 2: Divide array by calculating mid index i.e. $\text{mid} = l + (r - l)/2$

Step 3: Recursively find the maximum and minimum of left part by calling the same function i.e. $\text{left Minmax}[2] = \text{minmax}(X, l, \text{mid})$

Step 4: Recursively find the maximum and minimum for right part by calling the same function i.e. $\text{right Minmax}[2] = \text{minmax}(X, \text{mid} + 1, r)$

Step 5: Finally, get the overall maximum and minimum by comparing the min and max of both halves.

Step 6: Stop.

RESULT:

Thus, the C program for finding maximum and minimum numbers in a given list of n numbers using the divide and conquer technique was executed successfully and the output was verified.

12.Implement Merge sort and Quick sort methods to sort an array of elements and determine the time required to sort. Repeat the experiment for different values of n , the number of elements in the list to be sorted and plot a graph of the time taken versus n

AIM :

To write a C program to implement Merge sort and Quick sort methods to sort an array of elements and determine the time required to sort. Repeat the experiment for different values of n , the number of elements in the list to be sorted and plot a graph of the time taken versus n .

ALGORITHM (MERGE SORT):

step 1: start

step 2: declare array and left, right, mid variable

step 3: perform merge function.

if left > right return

mid= (left+right)/2 mergesort(array, left, mid) mergesort(array, mid+1, right)

merge(array, left, mid, right)

step 4: Stop

ALGORITHM (QUICK SORT):

Step 1: Create a recursive function (say quicksort()) to implement the quicksort.

Step 2: Partition the range to be sorted (initially the range is from 0 to $N-1$) and return the correct position of the pivot (say pi).

Select the rightmost value of the range to be the pivot.

Iterate from the left and compare the element with the pivot and perform the partition as shown above.

Return the correct position of the pivot.

Step 3: Recursively call the quicksort for the left and the right part of the pi .

RESULT:

Thus, the C program Merge sort and Quick sort methods for was executed successfully and the output was verified.

13.Implement N Queens problem using Backtracking

AIM :

To write a C program to implement N Queens problem using Backtracking.

ALGORITHM (MERGE SORT):

Step 1: Initialize an empty chessboard of size $N \times N$.

Step 2: Start with the leftmost column and place a queen in the first row of that column.

Step 3: Move to the next column and place a queen in the first row of that column.

Step 4: Repeat step 3 until either all N queens have been placed or it is impossible to place a queen in the current column without violating the rules of the problem.

Step 5: If all N queens have been placed, print the solution.

Step 6: If it is not possible to place a queen in the current column without violating the rules of the problem, backtrack to the previous column.

Step 7: Remove the queen from the previous column and move it down one row.

Step 8: Repeat steps 4–7 until all possible configurations have been tried.

RESULT:

Thus, the C program Merge sort and Quick sort methods was executed successfully and the output was verified.

14. Implement any scheme to find the optimal solution for the Traveling Salesperson problem and then solve the same problem instance using any approximation algorithm and determine the error in the approximation

AIM :

To write a C program to implement any scheme to find the optimal solution for the Traveling Salesperson problem and then solve the same problem instance using any approximation algorithm and determine the error in the approximation .

ALGORITHM:

Step 1: Start on an arbitrary vertex as current vertex.

Step 2: Find out the shortest edge connecting current vertex and an unvisited vertex V.

Step 3: Set current vertex to V.

Step 4: Mark V as visited.

Step 5: If all the vertices in domain are visited, then terminate.

Step 6: Go to step 2.

Step 7: The sequence of the visited vertices is the output of the algorithm.

RESULT:

Thus the C program to find the optimal solution for the Traveling Salesperson problem to find the optimal solution for the Traveling Salesperson problem was executed successfully and the output was verified.

15.Implement randomized algorithms for finding the kth smallest number

AIM :

To write a C program to implement randomized algorithms for finding the kth smallest number.

ALGORITHM:

Step 1: find the k'th element in A using 'selection algorithm', let it be 'z'

Step 2: initialize an empty list 'L'

Step 3: initialize counter←0

Step 4: for each element in A:

: if element < z:

: counter←counter + 1 ; L.add(element)

Step 5: for each element in A:

: if element == z AND count < k:

: counter←counter + 1 ; L.add(element)

Step 6: return L

RESULT:

Thus, the C program to implement randomized algorithms for finding the kth smallest number was executed successfully and the output was verified.